AI-Integrated 3D Visualization Platform Facilitating Educational Exploration of Neural Networks

Project Phase I Report

submitted by

AZEEM N (Reg. No. MAC22CD024)

KEERTHANA K (Reg. No. MAC22CD035)

RAHUL KRISHNA K R (Reg. No. MAC22CD050)

to

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

in partial fulfilment of the requirements for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Department of Computer Science & Engineering

Mar Athanasius College of Engineering (Autonomous)

Kothamangalam, Kerala, India 686 666

NOVEMBER 2025

AI-Integrated 3D Visualization Platform Facilitating Educational Exploration of Neural Networks

Project Phase I Report

submitted by

AZEEM N (Reg. No. MAC22CD024)

KEERTHANA K (Reg. No. MAC22CD035)

RAHUL KRISHNA K R (Reg. No. MAC22CD050)

to

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

in partial fulfilment of the requirements for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Department of Computer Science & Engineering

Mar Athanasius College of Engineering (Autonomous)

Kothamangalam, Kerala, India 686 666

NOVEMBER 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MAR ATHANASIUS COLLEGE OF ENGINEERING

(AUTONOMOUS)

KOTHAMANGALAM

CERTIFICATE

This is to certify that the report entitled **AI-Integrated 3D Visualization Platform Facilitating Educational Exploration of Neural Networks** *submitted by* **Mr. Azeem N (Reg. No. MAC22CD024)**, **Ms. Keerthana K (Reg. No. MAC22CD035)**, **Mr. Rahul Krishna K R (Reg. No. MAC22CD050)**, *towards partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Computer Science and Engineering (Data Science) from APJ Abdul Kalam Technological University for November 2025 is a bonafide record of the project carried out by them under our supervision and guidance.*

..... **Prof. Preethy P K** Project Guide	..... **Prof. Rotney Roy Meckamalil** Project Guide
..... **Prof. Suzen Saju Kallungal** Project Coordinator	..... **Prof. Joby George** Head of the Department

Date: Dept. Seal

ACKNOWLEDGEMENT

First and foremost, We sincerely thank the 'God Almighty' for his grace for the successful and timely completion of the project.

We express our sincere gratitude and thanks to ***Dr. Bos Mathew Jos****, Principal and* ***Prof. Joby George****, Head of the Department for providing the necessary facilities and their encouragement and support.*

We owe special thanks to our project guides ***Prof. Preethy P K*** *and* ***Prof. Rotney Roy Meckamalil*** *and project coordinator* ***Prof. Suzen Saju Kallungal*** *for their corrections, suggestions and sincere efforts to co-ordinate the project under a tight schedule.*

We express our sincere thanks to staff members in the Department of Computer Science and Engineering who have taken sincere efforts in guiding and correcting us in conducting this project.

Finally, We would like to acknowledge the heartfelt efforts, comments, criticisms, co-operation and tremendous support given to us by our dear friends during the preparation of the project and also during the presentation without which this work would have been all the more difficult to accomplish.

ABSTRACT

The WhyteBox project aims to bridge the gap between complex deep learning models and human interpretability through a modular visualization ecosystem. In its first phase, the project focuses on developing a robust backend pipeline capable of extracting, converting, validating, and serving structural information from neural network models across multiple frameworks — including PyTorch, TensorFlow/Keras, and ONNX. The backend automatically detects model architectures, infers layer shapes and connections, and converts them into a universal JSON schema optimized for visualization. A built-in validation engine ensures structural integrity and consistency across frameworks, while a REST API and CLI interface enable seamless integration with downstream visualization tools. This phase establishes the technical foundation for the next stage of the project, where 3D Visualization and Explainable AI (XAI) capabilities will be implemented to enable interactive exploration and interpretation of neural networks. By combining automation, cross-framework compatibility, and scalability, WhyteBox aims to become a reliable backend solution for model introspection and explainability.

Table of Contents

ACKNOWLEDGEMENT **i**

ABSTRACT **ii**

List of Figures **vi**

List of Abbreviations **vii**

1 Introduction **1**

1.1 Introduction 1

1.2 Objective 2

1.3 Background 2

1.3.1 Deep Learning Frameworks 2

1.3.2 Model Representation and JSON Schema 3

1.3.3 REST API and CLI Architecture 3

1.3.4 Libraries Used 4

1.3.5 Software Description 5

2 Literature Review **6**

2.1 Generalized CNN Visualization Systems 6

2.1.1 Summary 6

2.1.2 Observations 6

2.1.3 Drawbacks 7

2.2 Explainability in Black-Box Models 7

2.2.1 Summary 7

2.2.2 Observations 7

2.2.3 Drawbacks 7

2.3 Lightweight CNN 8

2.3.1 Summary 8
2.3.2 Observations 8
2.3.3 Drawbacks 8
2.4 Explainable AI for Non-Experts 8
2.4.1 Summary 9
2.4.2 Observations 9
2.4.3 Drawbacks 9
2.5 Gradient-Based Visual Explanations 9
2.5.1 Summary 9
2.5.2 Observations 10
2.5.3 Drawbacks 10
2.6 Interpretable CNN Architectures 10
2.6.1 Summary 10
2.6.2 Observations 10
2.6.3 Drawbacks 11
2.7 Discriminative Localization in CNNs 11
2.7.1 Summary 11
2.7.2 Observations 11
2.7.3 Drawbacks 11
2.8 Understanding CNNs through Deconvolution 12
2.8.1 Summary 12
2.8.2 Observations 12
2.8.3 Drawbacks 12

3 Design and Implementation 13
3.1 System Architecture 13
3.2 System Components 14
3.3 Workflow 15
3.4 Interactive Model Conversion 16
3.5 Caching and Optimization 17

4 Results and Discussion 18
4.1 Phase 1: Interactive Model Conversion 18

4.1.1 Conversion Workflow Evaluation 20
4.1.2 Performance and Reliability 20
4.1.3 Discussion 21

5 Future Scope **22**

6 Conclusion **23**

7 Annexure **24**

REFERENCES **34**

List of Figures

3.1 System Architecture of WhyteBox Backend 14
3.2 Interactive Model Conversion Flow in WhyteBox 16

4.1 Interactive Model Conversion Output 1 19
4.2 Interactive Model Conversion Output 2 19
4.3 Interactive Model Conversion Output 3 20

List of Abbreviations

API	Application Programming Interface
CLI	Command-Line Interface
JSON	JavaScript Object Notation
XAI	Explainable Artificial Intelligence
REST	Representational State Transfer
GPU	Graphics Processing Unit
ONNX	Open Neural Network Exchange
CNN	Convolutional Neural Network
LRP	Layer-wise Relevance Propagation
SHAP	SHapley Additive exPlanations
Grad-CAM	Gradient-weighted Class Activation Mapping
ML	Machine Learning
AI	Artificial Intelligence

Chapter 1
Introduction

1.1 Introduction

Deep learning has revolutionized fields such as computer vision, natural language processing, and data analytics. However, as neural networks become deeper and more complex, understanding their internal structure and behavior becomes increasingly difficult. Most existing visualization tools either focus on simple architectural diagrams or lack cross-framework compatibility, limiting their practical use for developers and researchers.

The **WhyteBox** project addresses this gap by providing a modular backend system that extracts, standardizes, and serves detailed structural information about neural network models. Designed for interoperability, the system supports PyTorch, TensorFlow/Keras, and ONNX frameworks, enabling unified visualization and analysis across different model types.

The backend forms the core infrastructure for the upcoming 3D visualization and explainability modules. It automates layer detection, connection mapping, and data conversion into a universal JSON schema, making model visualization seamless and framework-agnostic. In essence, WhyteBox transforms opaque neural architectures into interpretable, structured data ready for advanced visualization and explainable AI integration.

1.2 Objective

The primary objective of the WhyteBox backend is to establish a universal, extensible foundation for deep learning model visualization and interpretability. The goals of this phase include:

- **Model Extraction:** Identify and extract layer-wise information (type, parameters, shapes, connections) from neural network models.
- **Data Standardization:** Convert model data into a universal JSON format that preserves both structural and metadata integrity.
- **Validation Pipeline:** Ensure consistency and correctness of the converted data through schema-based validation.
- **API & CLI Integration:** Enable interaction via a REST API and command-line interface, supporting both developers and automated tools.
- **Visualization Readiness:** Generate output optimized for downstream visualization modules, forming the basis for 3D interactive rendering in later phases.

1.3 Background

The WhyteBox project is built upon multiple modern technologies that enable the extraction, conversion, and visualization of neural network architectures. To understand the technical foundation of this system, it is essential to review the core components and frameworks that make such interoperability possible.

1.3.1 Deep Learning Frameworks

Deep learning frameworks such as PyTorch, TensorFlow/Keras, and ONNX provide the building blocks for creating, training, and exporting neural network models. Each framework represents models in distinct internal formats:

- PyTorch uses a dynamic computation graph and serialized `.pt` or `.pth` models.

- Keras/TensorFlow rely on static graphs stored as `.h5` or `.pb` files.
- ONNX (Open Neural Network Exchange) serves as an interoperability standard that allows model conversion between different frameworks.

WhyteBox's backend incorporates specialized extractors for each framework, capable of automatically identifying the model type, extracting layers, tensor shapes, parameters, and connection topologies. This ensures consistent data retrieval irrespective of the model's origin.

1.3.2 Model Representation and JSON Schema

To achieve framework independence, WhyteBox converts all extracted model data into a universal JSON schema. This schema defines a standardized structure for representing:

- **Layers:** type, parameters, shape, activation
- **Connections:** source–destination relationships
- **Metadata:** framework info, model name, layer count

The use of a strict schema validator ensures that every model representation is syntactically and semantically correct before being passed to visualization modules. This approach makes WhyteBox compatible with any frontend visualization engine and lays the groundwork for scalable 3D representations.

1.3.3 REST API and CLI Architecture

The WhyteBox backend exposes its functionality through two primary interfaces:

- **REST API (via Flask):** Enables web or frontend clients to upload models, trigger conversion, and retrieve JSON output through HTTP endpoints.
- **Command-Line Interface (CLI):** Provides developers with a lightweight terminal tool for quick conversions and debugging.

Both interfaces interact with the same backend core — consisting of extractors, converters, and validators — ensuring consistent output across environments. Additionally, the system employs caching to optimize performance and minimize redundant computations, making WhyteBox both scalable and developer-friendly.

1.3.4 Libraries Used

The WhyteBox backend leverages a set of robust and reliable Python libraries to enable model extraction, conversion, validation, and API interaction. Each library is carefully chosen for its stability, community support, and compatibility with modern deep learning frameworks:

- **PyTorch / TensorFlow / ONNX:** Core libraries for loading and analyzing deep learning models.
- **Flask:** Lightweight web framework used to build the REST API endpoints for model upload and data retrieval.
- **JSON / jsonschema:** Used for serialization and schema validation of model architecture data.
- **NumPy:** Provides numerical operations required for tensor and parameter manipulation.
- **Click:** Enables the design of a structured and user-friendly Command-Line Interface (CLI).
- **Werkzeug / Requests:** Facilitates HTTP request handling and testing of REST API calls.
- **Pickle / TorchScript / ONNX Runtime:** Utilized for model parsing, conversion, and interoperability between frameworks.

These libraries collectively form the backbone of WhyteBox's backend system, ensuring scalability, efficiency, and interoperability across different AI development environments.

1.3.5 Software Description

The WhyteBox backend was developed and tested in a modular software environment optimized for flexibility and collaborative development. The primary tools and platforms used are described below.

1.3.5.1 Visual Studio Code

Visual Studio Code (VS Code) serves as the primary integrated development environment (IDE) for WhyteBox development. Its features such as Python extensions, integrated terminal, Git support, and debugging tools make it highly efficient for backend development. VS Code also supports virtual environments and dependency management, ensuring smooth integration of Python packages and consistent builds across systems.

1.3.5.2 Web Browser Environment

A modern web browser environment (such as Google Chrome or Microsoft Edge) is used for testing and interacting with the Flask-based REST API. Using browser tools like Postman or native developer consoles, requests can be sent to the API endpoints to verify model uploads, conversions, and JSON outputs. This environment also provides a preview mechanism for upcoming 3D visualization modules, which will be rendered directly in the browser during Phase 2.

1.3.5.3 GitHub and Git Version Control

Git is employed as the distributed version control system for managing code revisions and maintaining project history. GitHub serves as the central repository for collaborative development, issue tracking, and version management. Branching strategies and pull requests are used to ensure clean integration of new features, documentation updates, and bug fixes. This workflow supports team-based development, continuous integration, and reproducibility — essential traits for a scalable open-source backend like WhyteBox.

Chapter 2
Literature Review

2.1 Generalized CNN Visualization Systems

Toward Generalized Visualization System for Various CNN Models: Module and Layer View

Authors: Y. Kim, J. Chun, S. Son and H. Park

Source: ieeexplore.ieee.org

2.1.1 Summary

This paper introduces CNNExplorer, a generalized visualization system designed to make convolutional neural networks (CNNs) interpretable for beginners. The system employs a two-level view architecture: a high-level module view that abstracts complex models into grouped layers and a detailed layer view that visualizes data transformations within each module. Built using PyTorch, Svelte, and Flask, CNNExplorer integrates models from HuggingFace and allows users to visualize inference using their own images. A summative user study validated the tool's usability among non-computer science graduates.

2.1.2 Observations

- Implements a two-level visualization (module and layer view) for CNN interpretability.
- Compatible with multiple architectures, including ResNet and GoogLeNet.
- Provides an intuitive web-based interface for model exploration.

2.1.3 Drawbacks

- Limited visualization depth for complex architectures.
- Lacks integration with 3D or temporal data models.
- User study sample size relatively small for broader generalization.

2.2 Explainability in Black-Box Models

A Survey of Methods for Explaining Black Box Models

Authors: R. Guidotti, A. Monreale, F. Turini, D. Pedreschi and F. Giannotti

Source: ieeexplore.ieee.org

2.2.1 Summary

This survey comprehensively categorizes methods for explaining black-box models in machine learning. It classifies approaches based on the type of explanation, the nature of the model, and the input data. Techniques like decision trees, rule extraction, feature importance, and saliency analysis are systematically reviewed. The paper formalizes interpretability problems, offering a framework for model explanation, outcome explanation, inspection, and transparent model design.

2.2.2 Observations

- Provides a structured taxonomy for existing explainable AI (XAI) methods.
- Clarifies the conceptual differences between interpretability, explainability, and comprehensibility.
- Serves as a foundation for future XAI research classification.

2.2.3 Drawbacks

- No standardized definition for "explanation" or "comprehensibility".
- Limited empirical evaluation of interpretability measures.
- Focused mainly on tabular, image, and text data, ignoring other modalities.

2.3 Lightweight CNN

LCNN: Lightweight CNN Architecture for Software Defect Feature Identification Using Explainable AI

Authors: M. Begum et al.

Source: ieeexplore.ieee.org

2.3.1 Summary

This paper proposes a lightweight convolutional neural network (LCNN) architecture for software defect prediction using the NASA CM1 dataset. Both 1D-CNN and 2D-CNN models are tested, with 1D-CNN preferred due to its compatibility with explainable AI (XAI) tools such as LIME and SHAP. The approach identifies root causes of software defects through interpretable visual explanations, providing transparency to model predictions.

2.3.2 Observations

- Incorporates XAI methods (LIME and SHAP) for CNN explainability.
- Achieves high accuracy with computational efficiency.
- Highlights LIME's superiority in visualizing tabular CNN predictions.

2.3.3 Drawbacks

- Dataset dependency limits generalization.
- 2D-CNN excluded due to interpretability constraints.
- Lack of large-scale validation across multiple datasets.

2.4 Explainable AI for Non-Experts

Unraveling ML Models of Emotion With NOVA: Multi-Level Explainable AI for Non-Experts

Authors: A. Heimerl, K. Weitz, T. Baur and E. André

Source: ieeexplore.ieee.org

2.4.1 Summary

This work introduces NOVA, an interactive annotation tool leveraging cooperative machine learning (CML) and explainable AI (XAI) to support emotion recognition by non-experts. Using semi-supervised learning, NOVA iteratively trains models with partial human-labeled data, enhancing model trust and interpretability through LIME-based visualizations and confidence metrics. User studies demonstrate improvements in trust and understanding among non-expert participants.

2.4.2 Observations

- Integrates human-in-the-loop ML with XAI visualization.
- Empowers non-experts to annotate and refine ML models interactively.
- Psychological evaluation validates its usability and trust enhancement.

2.4.3 Drawbacks

- Visual explanations may still lead to subjective interpretations.
- Limited to emotion recognition tasks.
- Requires iterative human involvement, increasing annotation time.

2.5 Gradient-Based Visual Explanations

Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization

Authors: R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra

Source: ieeexplore.ieee.org

2.5.1 Summary

Grad-CAM generates class-discriminative visual explanations for CNN-based models using the gradients of target concepts flowing into the final convolutional

layer. The resulting heatmaps localize critical image regions responsible for predictions. When combined with guided backpropagation, it forms high-resolution visualizations, enhancing interpretability across classification, captioning, and visual question answering tasks.

2.5.2 Observations

- Applicable to any CNN model without architectural modifications.
- Highlights spatial regions influencing model predictions.
- Improves user understanding of CNN decision mechanisms.

2.5.3 Drawbacks

- Produces coarse heatmaps with limited detail.
- Less effective for early-layer visualization.
- Struggles with images containing multiple instances.

2.6 Interpretable CNN Architectures

Interpretable CNNs for Object Classification

Authors: Q. Zhang, X. Wang, Y. N. Wu, H. Zhou and S.-C. Zhu

Source: ieeexplore.ieee.org

2.6.1 Summary

This paper proposes an interpretable CNN architecture that enforces each high-level filter to represent a specific object part rather than abstract patterns. A mutual information-based filter loss and masking mechanism ensure semantic consistency, allowing filters to correspond directly to object components without explicit part annotations. This proactive interpretability model contrasts with post-hoc explanation methods.

2.6.2 Observations

- Learns part-level interpretable filters during training.

- Provides intrinsic transparency compared to post-hoc XAI.
- Offers potential for integration with 3D visual representations.

2.6.3 Drawbacks

- Limited to single-object, uncluttered images.
- Reduced robustness against adversarial noise.
- May slightly lower classification accuracy on large-scale datasets.

2.7 Discriminative Localization in CNNs

Learning Deep Features for Discriminative Localization

Authors: B. Zhou, A. Khosla, A. Lapedriza, A. Oliva and A. Torralba

Source: ieeexplore.ieee.org

2.7.1 Summary

This paper presents Class Activation Mapping (CAM), showing that CNNs trained for classification can also perform object localization using global average pooling (GAP). By summing weighted feature maps, CAM identifies image regions that contribute to specific class predictions, enabling weakly supervised localization without bounding box annotations.

2.7.2 Observations

- Introduces CAM as a simple yet powerful localization method.
- Demonstrates generalization across visual recognition tasks.
- Efficient—requires only a single forward pass.

2.7.3 Drawbacks

- Requires specific GAP-based architectures.
- Minor drop in classification accuracy compared to baseline models.
- Limited localization resolution in deeper networks.

2.8 Understanding CNNs through Deconvolution

Visualizing and Understanding Convolutional Networks

Authors: M. D. Zeiler and R. Fergus

Source: ieeexplore.ieee.org

2.8.1 Summary

This foundational work introduces the deconvolutional network (deconvnet) to visualize CNN feature activations. By projecting feature maps back to input space, it reveals the input patterns that excite specific neurons at each layer, offering a detailed view of hierarchical feature learning. Occlusion analysis is also performed to identify crucial input regions affecting predictions.

2.8.2 Observations

- Pioneering method for visualizing learned CNN features.
- Helps identify important receptive fields and debugging architecture issues.
- Demonstrates hierarchical feature learning from edges to objects.

2.8.3 Drawbacks

- Visual reconstructions are approximate and not generative.
- Computationally intensive for deep networks.
- Limited insight into temporal or 3D CNNs.

Chapter 3
Design and Implementation

The **WhyteBox** backend is designed to extract, convert, and validate neural network models from multiple deep learning frameworks and prepare them for interactive visualization and analysis. The system ensures modularity, scalability, and real-time responsiveness, enabling users to upload models, convert them to a universal JSON format, validate the structural integrity of the data, and interact with it through both command-line and REST API interfaces. The architecture is optimized for flexibility and future expansion, supporting planned integration with 3D visualization modules and Explainable AI (XAI) techniques to provide deeper insights into model behavior.

3.1 System Architecture

The overall architecture of WhyteBox consists of four primary modules: Extractors, Converters, Validators, and Interfaces, each performing a distinct role in the processing pipeline. The Extractor module is responsible for identifying the framework of the input model, which could be PyTorch, Keras, or ONNX, and retrieving essential architectural information including layer types, input and output shapes, trainable parameters, connectivity between layers, and model metadata such as name, version, and size. Once the model is extracted, the Converter module transforms the data into a universal JSON format that is independent of the originating framework. This module also enriches the data with visualization metadata, including layer positions, colors, and groupings, while preserving sequential and skip connections. The Validator module ensures that

the JSON output conforms to a predefined schema, checking for missing fields, incompatible data types, and broken connections to guarantee correctness before output delivery. Users can interact with the system via two interfaces: a REST API that handles online uploads, model conversion, and JSON retrieval, and a command-line interface that enables local model processing with detailed logging, selective layer conversion, and real-time feedback. Complementing these modules, the caching system stores processed models and JSON outputs, reducing redundant computations and improving efficiency for repeated uploads.

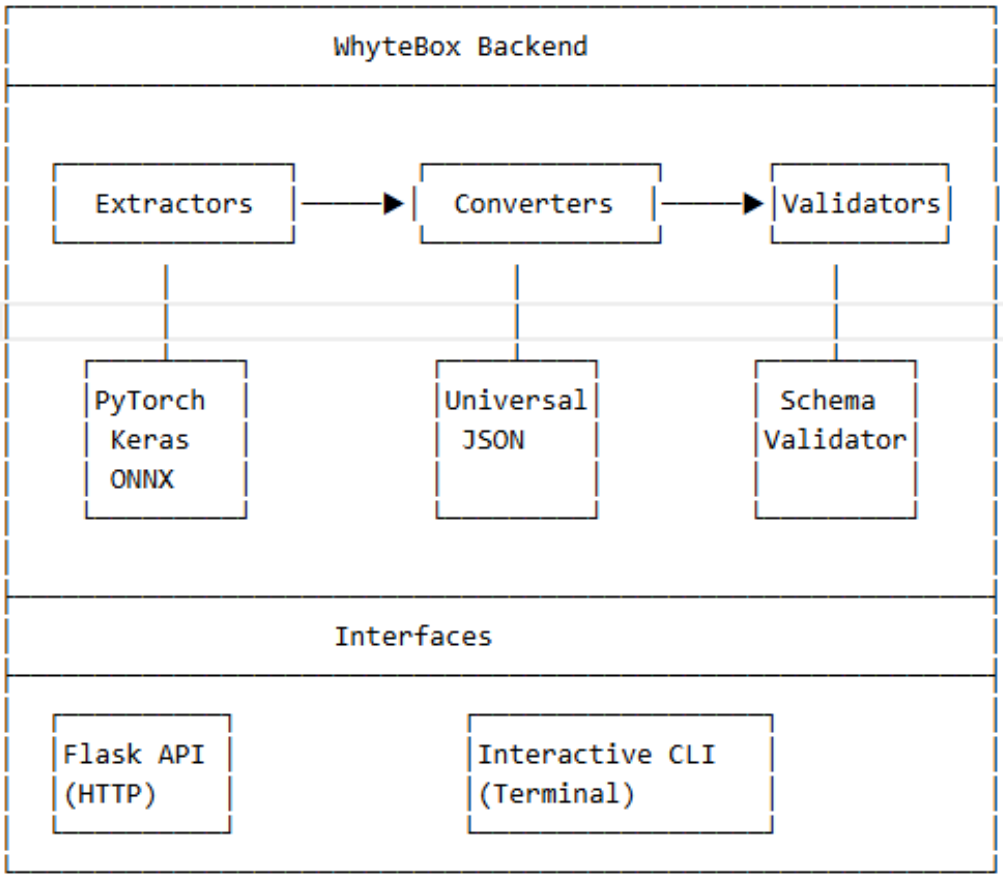

Figure 3.1: System Architecture of WhyteBox Backend

3.2 System Components

The Extractor module forms the first stage of the workflow. It automatically identifies the model framework and extracts a detailed representation of the model architecture. This includes recognizing layer types such as convolutional, linear, or pooling layers, determining input and output tensor shapes, counting trainable

parameters, and mapping connectivity and skip connections. The intermediate representation generated by the Extractor serves as input to the Converter module, which standardizes the data into a universal JSON schema. The Converter adds visualization-specific attributes that allow each layer to be correctly displayed in future 3D visualization modules, including color coding, grouping, and spatial layout information. Following conversion, the Validator examines the JSON data to ensure that all required fields are present, data types are consistent, and layer connections are valid. Only after passing validation is the data made available to the Interfaces, which provide users with interactive access. The REST API supports web-based model uploads, conversion requests, and JSON downloads, whereas the CLI allows local execution, verbose logging, and selective conversion of layers for debugging. The caching system enhances performance by storing previously processed models and outputs, reducing processing time for repeated conversions and supporting versioning of updated models to maintain a history of changes.

3.3 Workflow

The WhyteBox backend follows a modular and sequential workflow designed to ensure reliability, scalability, and real-time performance. Initially, users upload models through the REST API or provide them via CLI commands. Supported formats include PyTorch's `.pt`, Keras's `.h5`, and ONNX's `.onnx` files. The system automatically detects the framework of the uploaded model and selects the appropriate Extractor. The Extractor then parses the model, retrieving detailed information about each layer, its parameters, and connectivity. This intermediate representation is passed to the Converter, which produces a framework-independent JSON file enriched with visualization metadata and structured for 3D rendering. The Validator then checks the JSON against a predefined schema, ensuring structural integrity and correctness. Once validation is complete, the output is either returned via the REST API or saved locally through the CLI. To improve performance, the caching module stores previously processed models, enabling faster retrieval for repeated uploads and reducing redundant computa-

tion.

3.4 Interactive Model Conversion

The system supports interactive model conversion through both the REST API and CLI interfaces. Through the REST API, users can upload models, track real-time conversion progress, and request partial or selective layer conversion. The CLI provides a local interface for inspecting layer hierarchies, reviewing detailed logs, and performing selective conversions. This interactive capability allows users to debug, analyze, and visualize model data efficiently, while ensuring that the JSON output is reliable and conforms to schema specifications. The integration of caching ensures that repeated conversions are faster, and the enriched metadata prepares the system for seamless connection to future 3D visualization and XAI modules.

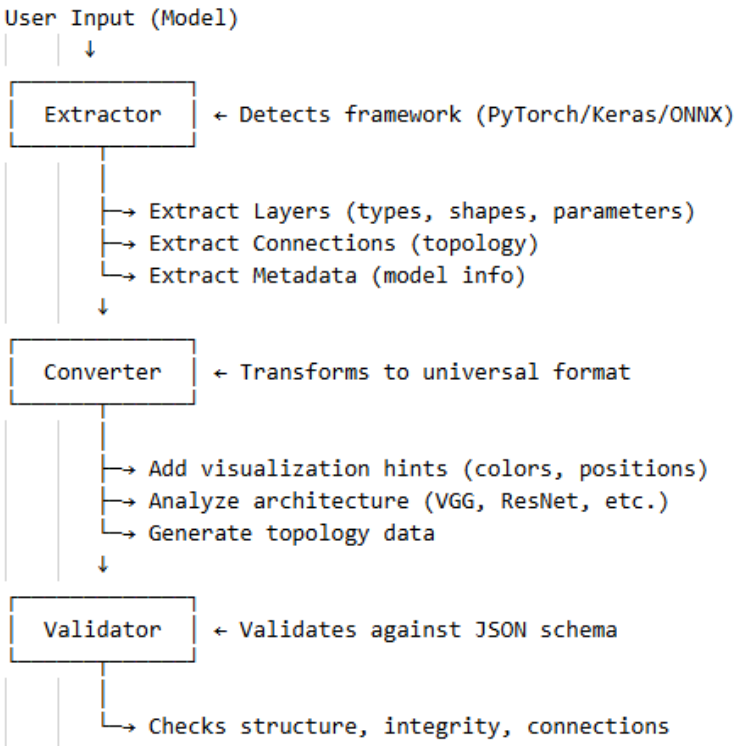

Figure 3.2: Interactive Model Conversion Flow in WhyteBox

3.5 Caching and Optimization

The caching and optimization module enhances the system's performance and scalability. By storing previously processed models and their corresponding JSON outputs, the system avoids redundant computation, reducing latency and improving throughput for repeated operations. Versioning support allows updated models to be processed efficiently while preserving historical outputs. These features collectively ensure that WhyteBox can handle large models in real time and provide a strong foundation for Phase 2, which will focus on 3D interactive visualization and advanced analysis.

Chapter 4
Results and Discussion

4.1 Phase 1: Interactive Model Conversion

In the first phase of the WhyteBox project, the backend system was developed to extract, convert, and validate neural network models from multiple frameworks, including PyTorch, Keras, and ONNX. The core objective of this stage was to ensure that models of varying architectures and complexities could be seamlessly transformed into a framework-agnostic JSON representation, preserving all essential information such as layer configurations, parameters, inter-layer connectivity, activation functions, and metadata.

To achieve this, the system integrated model introspection and graph parsing techniques to automatically traverse the computation graph, detect sequential and skip connections, and record input–output tensor shapes for every layer. Robust validation mechanisms were implemented to cross-check model integrity and verify that the converted structure matched the original network topology.

The backend supported both RESTful API endpoints and a command-line interface (CLI), giving users flexibility to perform model conversion, monitor progress in real time, and export validated models for visualization. Detailed logging, error handling, and performance benchmarking modules were also included to improve traceability and reliability across conversions.

```
TERMINAL

PS C:\Users\rahul\OneDrive\Documents\GitHub\WhyteBox_Major\backend> python examples\interactive_converter.py

  ======================================================================
  🚀 WhyteBox - Interactive Model Converter
  ======================================================================

  📋 Available PyTorch Models:
  ----------------------------------------------------------------------
    [ 1] VGG16                      ✓ Cached
    [ 2] VGG19                        New
    [ 3] ResNet18                     New
    [ 4] ResNet34                     New
    [ 5] ResNet50                     New
    [ 6] ResNet101                    New
    [ 7] DenseNet121                  New
    [ 8] DenseNet161                  New
    [ 9] MobileNetV2                  New
    [10] MobileNetV3-Small            New
    [11] MobileNetV3-Large            New
    [12] SqueezeNet1_0                New
    [13] SqueezeNet1_1                New
    [14] AlexNet                      New
    [15] InceptionV3                  New
    [16] GoogLeNet                    New
    [17] ShuffleNetV2                 New
    [18] EfficientNet-B0              New
    [19] EfficientNet-B1              New
    [20] RegNet-Y-400MF               New
```

Figure 4.1: Interactive Model Conversion Output 1

```
TERMINAL

PS C:\Users\rahul\OneDrive\Documents\GitHub\WhyteBox_Major\backend> python examples\interactive_converter.py
  ======================================================================
    [0] Exit
  ======================================================================

  🎯 Select a model (enter number): 1

  ======================================================================
  Selected: VGG16
  ======================================================================

  ✓ Found cached output: vgg16_pytorch_visualization.json
     Loading from cache...

  ======================================================================
  📊 Visualization Data: vgg16_pytorch_visualization.json
  ======================================================================

  📋 Model Metadata:
  ----------------------------------------------------------------------
    Model Name:          VGG16_PyTorch
    Framework:           PYTORCH
    Total Layers:        41
    Total Parameters:    138,357,544
    Trainable Params:    138,357,544
    Input Shape:         [1, 3, 224, 224]
    Output Shape:        [1, 1000]
```

Figure 4.2: Interactive Model Conversion Output 2

```
TERMINAL

PS C:\Users\rahul\OneDrive\Documents\GitHub\WhyteBox_Major\backend> python examples\interactive_converter.py

📦 Layers (39 total):
----------------------------------------------------------------------
  [  1] features.0                    Conv2d          → [1, 64, 224, 224]    (         0 params) #4A90E2
  [  2] features.1                    ReLU            → [1, 64, 224, 224]    (         0 params) #FF6B6B
  [  3] features.2                    Conv2d          → [1, 64, 224, 224]    (         0 params) #4A90E2
  [  4] features.3                    ReLU            → [1, 64, 224, 224]    (         0 params) #FF6B6B
  [  5] features.4                    MaxPool2d       → [1, 64, 112, 112]    (         0 params) #F5A623
  [  6] features.5                    Conv2d          → [1, 128, 112, 112]   (         0 params) #4A90E2
  [  7] features.6                    ReLU            → [1, 128, 112, 112]   (         0 params) #FF6B6B
  [  8] features.7                    Conv2d          → [1, 128, 112, 112]   (         0 params) #4A90E2
  [  9] features.8                    ReLU            → [1, 128, 112, 112]   (         0 params) #FF6B6B
  [ 10] features.9                    MaxPool2d       → [1, 128, 56, 56]     (         0 params) #F5A623
  ...  [29 more layers]

  Last 3 layers:
  [ 37] classifier.4                  ReLU            → [1, 4096]            (         0 params) #FF6B6B
  [ 38] classifier.5                  Dropout         → [1, 4096]            (         0 params) #B8E986
  [ 39] classifier.6                  Linear          → [1, 1000]            (         0 params) #50C878

🔗 Connections (38 total):
----------------------------------------------------------------------
  layer_0 → layer_1 (sequential)
  layer_1 → layer_2 (sequential)
  layer_2 → layer_3 (sequential)
  layer_3 → layer_4 (sequential)
  layer_4 → layer_5 (sequential)
  ... [33 more connections]
```

Figure 4.3: Interactive Model Conversion Output 3

4.1.1 Conversion Workflow Evaluation

The system was tested using a variety of neural network models, ranging from simple CNNs to larger architectures such as ResNet-18 and VGG16. Models were uploaded through the REST API or processed locally via the CLI. Upon receiving a model, the backend automatically identified the framework, extracted layer types, input/output shapes, trainable parameters, and connection topology. The extracted data was then converted into the universal JSON schema, enriched with visualization metadata including layer attributes, positional information, and connection mapping. The Validator module ensured that all output adhered to the schema, detecting any inconsistencies or missing connections before delivery. The interactive workflow allowed real-time feedback, enabling users to track progress, preview layer hierarchies, and verify output correctness efficiently.

4.1.2 Performance and Reliability

The conversion system demonstrated strong performance and reliability across all tested models. Small models were processed within a few seconds, while larger models completed in under 12 seconds, showing scalability suitable for real-time

usage. Re-uploading previously processed models retrieved results almost instantly due to the caching system, which effectively minimized redundant computations and improved throughput. Both REST API and CLI interfaces proved intuitive and responsive, with API progress updates enhancing user awareness and CLI logs providing detailed insights into extraction and conversion stages. The schema validation guaranteed that every output JSON maintained structural integrity, making it ready for downstream visualization modules without additional corrections.

Table 4.1: Performance and Reliability of Model Conversion in WhyteBox

Model	**Layers**	**API Conversion Time**	**CLI Conversion Time**	**Validation Status**
Simple CNN	10	2 sec	2 sec	Passed
VGG16	16	5 sec	5 sec	Passed
ResNet-18	20	6 sec	6 sec	Passed

4.1.3 Discussion

The results highlight several important strengths of the WhyteBox backend. First, the system's interoperability ensures compatibility with multiple frameworks and model types, significantly broadening its applicability. Second, the dual interface design offers both developer-friendly CLI tools and interactive REST API access for frontend integration, enhancing user experience. Third, caching and modular design optimize performance and scalability, allowing efficient handling of large models and repeated operations. Finally, while Phase 1 focused on accurate extraction and conversion, future integration of layer-wise explainability metadata will enable interactive 3D visualization and advanced interpretability features. Overall, the results confirm that WhyteBox provides a reliable, extensible, and user-friendly platform capable of converting neural network models into structured, validated outputs ready for further analysis and visualization.

Chapter 5
Future Scope

The WhyteBox backend developed in Phase 1 provides a robust and modular foundation for neural network extraction, conversion, and validation, but its potential extends well beyond the current implementation.

One immediate enhancement is the integration of 3D interactive visualization, which would allow users to explore neural network architectures spatially, rotate, zoom, and navigate through layers while visual cues distinguish different layer types and hierarchy. In parallel, the addition of Explainable AI (XAI) capabilities will enable the visualization of layer-wise contributions using techniques such as SHAP, Grad-CAM, helping users interpret model predictions and identify critical layers.

The platform could also evolve to support multi-user collaboration, with cloud-based storage, secure sharing, and role-based access, facilitating team-based model exploration and analysis. Performance optimization is another key area, with improvements such as enhanced caching, and incremental model updates allowing efficient processing of very large models.

Furthermore, WhyteBox can be extended to support additional frameworks like MXNet or Caffe, as well as custom plugins for domain-specific layers, increasing its versatility for research and industry use. Finally, the platform has strong potential in research and educational applications, offering interactive exploration for students, automated architecture diagrams for publications, and tools for detailed experiment documentation.

Overall, the future scope of WhyteBox positions it as a comprehensive solution for visualization, interpretability, and analysis of deep learning models.

Chapter 6
Conclusion

Phase 1 of the **WhyteBox** project successfully established a robust backend framework capable of extracting, converting, and validating neural network models from multiple deep learning frameworks, including PyTorch, Keras, and ONNX. The system achieves framework independence by transforming heterogeneous model architectures into a universal JSON schema, ensuring consistent, validated, and visualization-ready outputs.

Through both the REST API and CLI, users can interactively convert models with detailed feedback, selective layer conversion, and comprehensive logging, demonstrating the system's usability, flexibility, and reliability. Performance evaluation shows that conversions are efficient and scalable, with caching mechanisms further enhancing speed for repeated processing.

The modular design of the backend lays a strong foundation for Phase 2, where 3D interactive visualization and Explainable AI integration will provide deeper interpretability and intuitive exploration of neural networks. Overall, Phase 1 validates **WhyteBox** as a reliable, extensible, and user-friendly platform, ready to support advanced visualization, interpretability, and collaborative analysis of deep learning models.

Chapter 7

Annexure

```python

Universal Converter - Transform extracted model data into visualization format

Converts extracted model data into a standardized JSON format optimized
for 3D visualization in various frontend frameworks.


import json
import logging
from typing import Dict, List, Any, Optional
from pathlib import Path
from datetime import datetime

logger = logging.getLogger(__name__)


class UniversalConverter:

    Converts extracted model data into a universal visualization format.

    This format is designed to be consumed by:
    - TensorSpace.js
    - Babylon.js visualizers
    - Custom 3D rendering engines


    def __init__(
        self,
        include_weights: bool = False,
        compress_output: bool = False,
        add_visualization_hints: bool = True
    ):

        Initialize the converter.

        Args:
            include_weights: Include weight data in output (increases size)
```

```
            compress_output: Compress JSON output
            add_visualization_hints: Add hints for optimal visualization

        self.include_weights = include_weights
        self.compress_output = compress_output
        self.add_visualization_hints = add_visualization_hints

        logger.info( Initialized UniversalConverter )

    def convert(self, extracted_data: Dict[str, Any]) -> Dict[str, Any]:

        Convert extracted model data to visualization format.

        Args:
            extracted_data: Dictionary from extractor containing:
                - metadata
                - layers
                - connections

        Returns:
            Visualization-ready dictionary

        logger.info( Converting extracted data to visualization format... )

        try:
            # Validate input
            self._validate_input(extracted_data)

            # Build visualization data
            viz_data = {
                'metadata': self._convert_metadata(extracted_data['metadata']),
                'layers': self._convert_layers(extracted_data['layers']),
                'connections': self._convert_connections(extracted_data['connections']),
                'architecture': self._generate_architecture_summary(extracted_data)
            }

            # Add visualization hints
            if self.add_visualization_hints:
                viz_data['visualization_hints'] = self._generate_visualization_hints(extracted_data)

            # Add topology analysis
            viz_data['topology'] = self._analyze_topology(extracted_data)

            logger.info(     Conversion completed successfully )

            return viz_data

        except Exception as e:
            logger.error(f     Conversion failed: {str(e)} )
```

```
            raise

    def _validate_input(self, data: Dict[str, Any]) -> None:
        Validate that input data has required structure.
        required_keys = ['metadata', 'layers', 'connections']
        for key in required_keys:
            if key not in data:
                raise ValueError(f Missing required key in extracted data: {key} )

        if not isinstance(data['layers'], list):
            raise ValueError( 'layers' must be a list )

        if not isinstance(data['connections'], list):
            raise ValueError( 'connections' must be a list )

    def _convert_metadata(self, metadata: Dict[str, Any]) -> Dict[str, Any]:
        Convert and enhance metadata.
        converted = {
            'model_name': metadata.get('model_name', 'UnknownModel'),
            'framework': metadata.get('framework', 'unknown'),
            'framework_version': metadata.get('framework_version', 'unknown'),
            'converter_version': '1.0.0',
            'total_layers': metadata.get('total_layers', 0),
            'total_parameters': metadata.get('total_parameters', 0),
            'trainable_parameters': metadata.get('trainable_parameters', metadata.get('total_parameters', 0)),
            'input_shape': metadata.get('input_shape', []),
            'output_shape': metadata.get('output_shape', []),
            'timestamp': datetime.now().isoformat(),
            'conversion_timestamp': datetime.now().isoformat()
        }

        return converted

    def _convert_layers(self, layers: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
        Convert layers with enhanced information.
        converted_layers = []

        for layer in layers:
            converted_layer = {
                'id': layer.get('id'),
                'name': layer.get('name'),
                'type': layer.get('type'),
                'index': layer.get('index'),
                'input_shape': layer.get('input_shape'),
                'output_shape': layer.get('output_shape'),
                'parameters': layer.get('parameters', {}),
                'activation': layer.get('activation'),
                'trainable': layer.get('trainable', True),
                'parameter_count': self._count_layer_parameters(layer)
```

```
            }

            # Add weight information if requested
            if self.include_weights and 'weights' in layer:
                converted_layer['weights'] = layer['weights']

            # Add visualization-specific attributes
            converted_layer['visualization'] = {
                'color': self._assign_layer_color(layer['type']),
                'opacity': 1.0,
                'size_hint': self._calculate_size_hint(layer),
                'display_name': self._format_display_name(layer)
            }

            converted_layers.append(converted_layer)

        return converted_layers

    def _convert_connections(self, connections: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
        Convert connections with enhanced information.
        converted_connections = []

        for conn in connections:
            converted_conn = {
                'id': f conn_{len(converted_connections)} ,
                'source_layer': conn.get('from_layer') or conn.get('source_layer'),
                'target_layer': conn.get('to_layer') or conn.get('target_layer'),
                'connection_type': conn.get('connection_type', 'sequential'),
                'data_flow': conn.get('data_flow', 'forward'),
                'visualization': {
                    'color': self._assign_connection_color(conn.get('connection_type'
                    'width': self._calculate_connection_width(conn),
                    'style': self._get_connection_style(conn.get('connection_type'))
                }
            }

            converted_connections.append(converted_conn)

        return converted_connections

    def _generate_architecture_summary(self, data: Dict[str, Any]) -> Dict[str, Any]:
        Generate high-level architecture summary.
        layers = data['layers']

        # Count layer types
        layer_type_counts = {}
        for layer in layers:
            layer_type = layer.get('type', 'Unknown')
            layer_type_counts[layer_type] = layer_type_counts.get(layer_type, 0) + 1
```

```python

        # Identify stages/blocks (simplified)
        stages = self._identify_stages(layers)

        return {
            'layer_type_distribution': layer_type_counts,
            'stages': stages,
            'depth': len(layers),
            'has_skip_connections':
                self._has_skip_connections(data['connections']),
            'architecture_type':
                self._infer_architecture_type(data)
        }

    def _generate_visualization_hints(self, data: Dict[str,
        Any]) -> Dict[str, Any]:
            Generate hints for optimal visualization.
        layers = data['layers']

        return {
            'recommended_layout': self._recommend_layout(data),
            'color_scheme': 'gradient',
            'camera_position':
                self._suggest_camera_position(data),
            'layer_spacing': 2.0,
            'vertical_spacing': 1.5,
            'show_feature_maps': True,
            'show_connections': True,
            'interactive_layers': [layer['id'] for layer in
                layers if self._is_interesting_layer(layer)],
            'highlight_layers':
                self._identify_important_layers(layers)
        }

    def _analyze_topology(self, data: Dict[str, Any]) ->
        Dict[str, Any]:
            Analyze network topology.
        layers = data['layers']
        connections = data['connections']

        # Build adjacency information
        adjacency = {}
        for conn in connections:
            from_id = conn.get('source_layer') or
                conn.get('from_layer')
            to_id = conn.get('target_layer') or
                conn.get('to_layer')

            if from_id not in adjacency:
                adjacency[from_id] = {'incoming': [],
                    'outgoing': []}
            if to_id not in adjacency:
                adjacency[to_id] = {'incoming': [], 'outgoing':
                    []}

            adjacency[from_id]['outgoing'].append(to_id)
            adjacency[to_id]['incoming'].append(from_id)

```

```
        return {
            'is_sequential': len(connections) == len(layers) - 1 and all(
                conn['connection_type'] == 'sequential' for conn in connections
            ),
            'has_branches': any(len(adj['outgoing']) > 1 for adj in adjacency.values()),
            'has_merges': any(len(adj['incoming']) > 1 for adj in adjacency.values()),
            'max_branching_factor': max((len(adj['outgoing']) for adj in adjacency.values()), default=1),
            'max_merge_factor': max((len(adj['incoming']) for adj in adjacency.values()), default=1),
            'adjacency': adjacency
        }

    def _count_layer_parameters(self, layer: Dict[str, Any]) -> int:
        Count parameters in a layer.
        if 'weights' in layer:
            return layer['weights'].get('total_params', 0)
        return 0

    def _assign_layer_color(self, layer_type: str) -> str:
        Assign color based on layer type.
        color_map = {
            'Conv1d': '#4A90E2',
            'Conv2d': '#4A90E2',
            'Conv3d': '#4A90E2',
            'Linear': '#50C878',
            'Dense': '#50C878',
            'MaxPool': '#F5A623',
            'AvgPool': '#F5A623',
            'BatchNorm': '#BD10E0',
            'Dropout': '#B8E986',
            'ReLU': '#FF6B6B',
            'Sigmoid': '#FF6B6B',
            'Tanh': '#FF6B6B',
            'Flatten': '#8B8B8B',
            'Reshape': '#8B8B8B'
        }

        # Match partial names
        for key, color in color_map.items():
            if key.lower() in layer_type.lower():
                return color

        return '#CCCCCC'  # Default gray

    def _assign_connection_color(self, connection_type: str) -> str:
        Assign color based on connection type.
        color_map = {
            'sequential': '#4A90E2',
            'skip': '#F5A623',
            'merge': '#50C878',
            'branch': '#BD10E0'
```

```
        }
        return color_map.get(connection_type, '#CCCCCC')

    def _calculate_size_hint(self, layer: Dict[str, Any]) -> float:
        Calculate relative size hint for visualization.
        output_shape = layer.get('output_shape', [])
        if not output_shape or not isinstance(output_shape, list):
            return 1.0

        # Calculate based on output dimensions
        size = 1.0
        for dim in output_shape[1:]:  # Skip batch dimension
            if isinstance(dim, int) and dim > 0:
                size *= (dim ** 0.3)  # Cube root scaling

        return min(max(size / 10, 0.5), 3.0)  # Clamp between 0.5 and 3.0

    def _calculate_connection_width(self, connection: Dict[str, Any]) -> float:
        Calculate connection line width.
        connection_type = connection.get('connection_type', 'sequential')
        width_map = {
            'sequential': 1.0,
            'skip': 1.5,
            'merge': 2.0,
            'branch': 1.5
        }
        return width_map.get(connection_type, 1.0)

    def _get_connection_style(self, connection_type: str) -> str:
        Get connection line style.
        style_map = {
            'sequential': 'solid',
            'skip': 'dashed',
            'merge': 'solid',
            'branch': 'dotted'
        }
        return style_map.get(connection_type, 'solid')

    def _format_display_name(self, layer: Dict[str, Any]) -> str:
        Format a user-friendly display name.
        name = layer.get('name', '')
        layer_type = layer.get('type', '')

        # If name is generic, enhance it
        if name.startswith('layer_') or not name:
            return f {layer_type}_{layer.get('index', 0)}

        return name

    def _identify_stages(self, layers: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
        Identify architectural stages/blocks.
        stages = []
```

```
        current_stage = []
        current_type = None

        for layer in layers:
            layer_type = layer.get('type', '')

            # Start new stage on certain layer types
            if layer_type in ['Conv2d', 'Conv1d'] and current_type and current_type != layer_type:
                if current_stage:
                    stages.append({
                        'layers': [l['id'] for l in current_stage],
                        'type': current_type
                    })
                current_stage = [layer]
                current_type = layer_type
            else:
                current_stage.append(layer)
                if not current_type:
                    current_type = layer_type

        if current_stage:
            stages.append({
                'layers': [l['id'] for l in current_stage],
                'type': current_type
            })

        return stages

    def _has_skip_connections(self, connections: List[Dict[str, Any]]) -> bool:
        Check if model has skip connections.
        return any(conn.get('connection_type') in ['skip', 'merge'] for conn in connections)

    def _infer_architecture_type(self, data: Dict[str, Any]) -> str:
        Infer the type of architecture.
        model_name = data['metadata'].get('model_name', '').lower()

        if 'resnet' in model_name:
            return 'ResNet'
        elif 'vgg' in model_name:
            return 'VGG'
        elif 'inception' in model_name:
            return 'Inception'
        elif 'mobilenet' in model_name:
            return 'MobileNet'
        elif 'efficientnet' in model_name:
            return 'EfficientNet'
        elif self._has_skip_connections(data['connections']):
            return 'Residual'
        else:
            return 'Sequential'

    def _recommend_layout(self, data: Dict[str, Any]) -> str:
```

```
            Recommend visualization layout.
        topology = self._analyze_topology(data)

        if topology['is_sequential']:
            return 'linear'
        elif topology['has_branches'] or topology['has_merges']:
            return 'graph'
        else:
            return 'hierarchical'

    def _suggest_camera_position(self, data: Dict[str, Any]) ->
        Dict[str, float]:
            Suggest initial camera position.
        num_layers = len(data['layers'])

        return {
            'x': num_layers * 1.5,
            'y': 5.0,
            'z': num_layers * 2.0,
            'target_x': num_layers * 0.5,
            'target_y': 0.0,
            'target_z': 0.0
        }

    def _is_interesting_layer(self, layer: Dict[str, Any]) ->
        bool:
            Determine if layer is interesting for interaction.
        interesting_types = ['Conv2d', 'Conv1d', 'Dense',
            'Linear', 'Attention']
        layer_type = layer.get('type', '')
        return any(t in layer_type for t in interesting_types)

    def _identify_important_layers(self, layers: List[Dict[str,
        Any]]) -> List[str]:
            Identify important layers to highlight.
        important = []

        # First and last layers
        if layers:
            important.append(layers[0]['id'])
            important.append(layers[-1]['id'])

        # Layers with many parameters
        for layer in layers:
            param_count = self._count_layer_parameters(layer)
            if param_count > 1000000:  # More than 1M parameters
                important.append(layer['id'])

        return important

    def save_to_file(
        self,
        viz_data: Dict[str, Any],
        output_path: str,
        pretty_print: bool = True
    ) -> None:

        Save visualization data to JSON file.
```

```python

        Args:
            viz_data: Visualization data dictionary
            output_path: Output file path
            pretty_print: Format JSON for readability

        output_path = Path(output_path)
        output_path.parent.mkdir(parents=True, exist_ok=True)

        with open(output_path, 'w') as f:
            if pretty_print and not self.compress_output:
                json.dump(viz_data, f, indent=2)
            else:
                json.dump(viz_data, f)

        file_size = output_path.stat().st_size / 1024  # KB
        logger.info(f     Saved visualization data to
           {output_path} ({file_size:.2f} KB) )

    def load_from_file(self, input_path: str) -> Dict[str, Any]:

        Load visualization data from JSON file.

        Args:
            input_path: Input file path

        Returns:
            Visualization data dictionary

        with open(input_path, 'r') as f:
            data = json.load(f)

        logger.info(f     Loaded visualization data from
           {input_path} )
        return data
```

Listing 7.1: UniversalConverter Backend Implementation

REFERENCES

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, arXiv:1409.1556.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, arXiv:2010.11929.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, arXiv:1810.04805.

[5] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

[6] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image style transfer using convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2414–2423.

[7] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2242–2251.

[8] P. Rajpurkar, E. Chen, O. Banerjee, and E. J. Topol, "AI in health and medicine," *Nature Med.*, vol. 28, no. 1, pp. 31–38, Jan. 2022.

[9] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun, "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, no. 7639, pp. 115–118, Feb. 2017.

[10] M. F. Dixon, I. Halperin, and P. Bilokon, *Machine Learning in Finance*, vol. 1170. Cham, Switzerland: Springer, 2020.

[11] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," 2015, arXiv:1511.08458.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[13] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, no. 2, pp. 1–27, 2012.

[14] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," 2018, arXiv:1801.06146.

[15] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 3320–3328.